

1. Objective

Refactoring a Layered Architecture into a **Modular Monolith** to enforce strict boundaries and reduce tight coupling between modules.

2. Architectural Implementation

- **Information Hiding:** Internal classes like `ProductRepository` and `CatalogServiceImpl` were set to package-private to hide implementation details.
- **Interface-Based Communication:** The `Orders` module interacts with the `Catalog` module solely through the `CatalogService` interface, without direct database access.
- **Factory Pattern:** Implemented `CatalogFactory` and `OrdersFactory` to manage component instantiation and dependency wiring securely.

3. Key Findings

- **Decoupling:** Modules are now independent; changing the internal logic of the `Catalog` does not affect the `Orders` module.
- **Data Integrity:** Stock reduction is handled through a service-to-service call, ensuring business rules are followed before an order is saved.

4. Conclusion

The lab successfully demonstrated how modular boundaries and public APIs improve system maintainability and encapsulation in a Java environment.